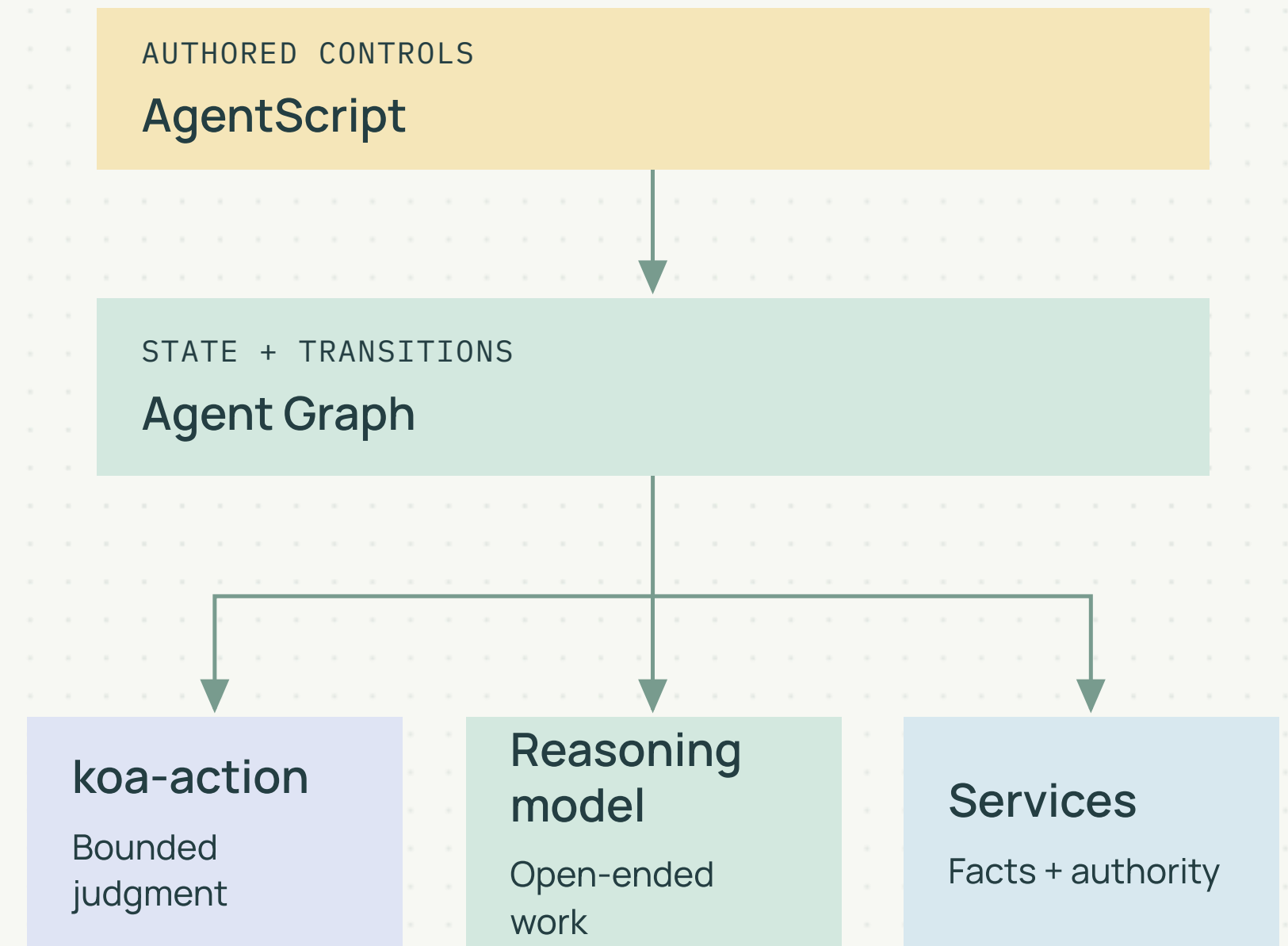


Disaggregated intelligence

Enterprise reasoning with
koa-action, Agent Graph
and AgentScript.

Specialized decisions, explicit workflows,
and reasoning where the case needs it.



Four benefits. One service to measure.

CORRECT OUTCOMES



Reliability

Test decisions and enforce required checks.

CONTROLLED VARIATION



Predictability

Bound outputs, paths, and recovery behavior.

LESS TOTAL WORK



Lower cost

Avoid expensive work that the case does not need.

SHORTER CRITICAL PATH



Faster inference

Use short decision steps on frequent routes.

One customer, two obligations

“Why was I charged twice,
and why is my service
down?”

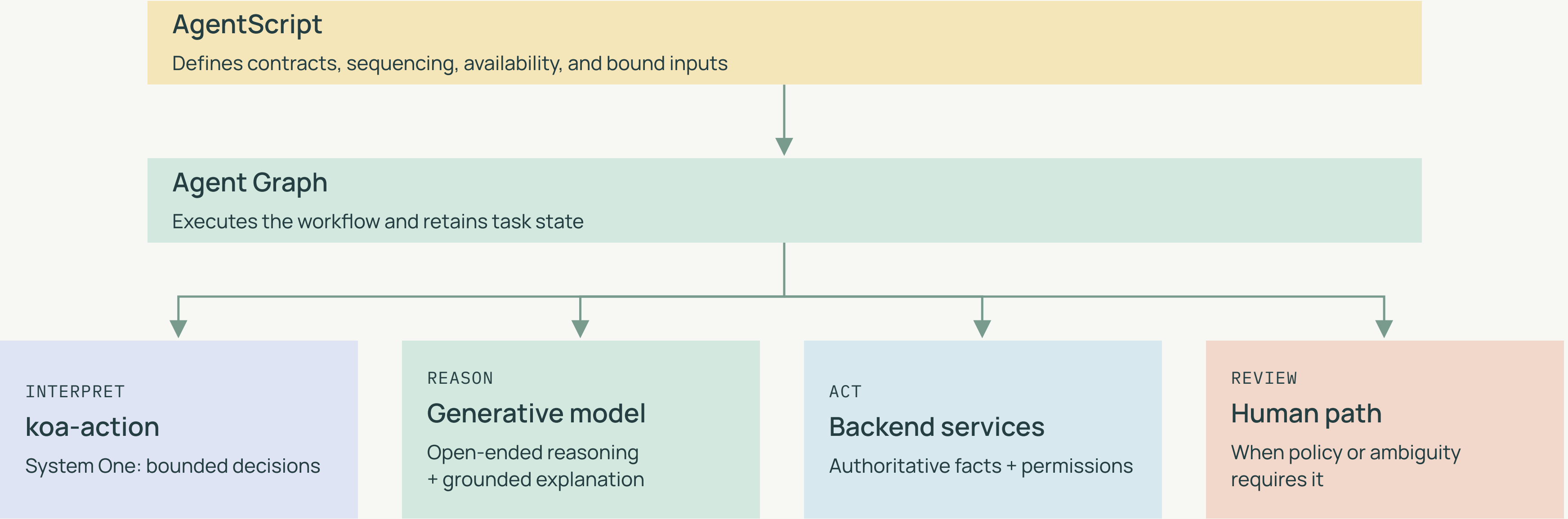
One message creates two obligations.

The case ledger

Invoice	Requested	Pending
Outage	Requested	Pending

What must survive
every transition?

Give each responsibility an owner

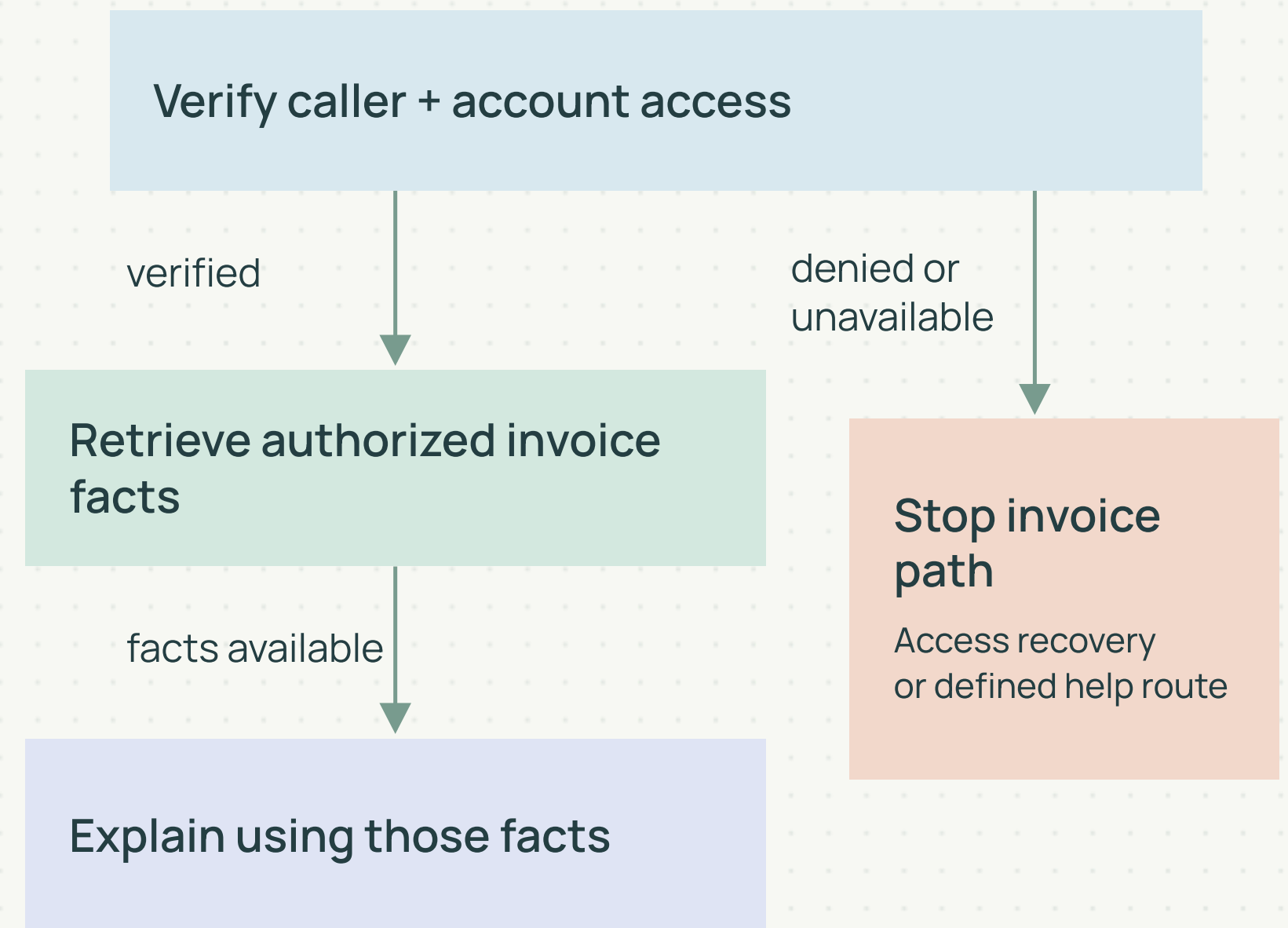


Process knowledge becomes topology

A PROCESS BURIED IN INSTRUCTIONS

Before explaining an invoice, verify account access. Retrieve the invoice only when access is allowed. Keep the outage pending.

Definitions and evidence still belong in node context.



A runtime sits beneath the harness

Harness

The agent loop, context strategy, tools, and operating policy.

Example in LangChain's stack: Deep Agents.

Runtime

State transitions and the execution of the configured workflow.

Agent Graph is analogous at this layer to LangGraph.

Configuration

The authored rules and bindings that shape behavior.

AgentScript configures Salesforce's runtime.

Start with a decision contract

QUESTION + INPUT

Which service route
is supported now?

Conversation so far
+ service category definitions.

APPLICATION LABELS · BOUNDARY EXAMPLES

“Charged twice.”	billing
“Charged twice; service down.”	mixed
“Something is wrong.”	unclear

Allowed answers

- billing
- technical
- account_access
- mixed
- unclear

Valid but unclear

Ask within a clarification budget.

Timeout or invalid output

Bound retries; then hand off safely.

Neither path authorizes a transaction.

Four bounded decisions, four contracts

01

Classification

Which route?

Select a category.

02

Noul

Cancellation requested?

Assess a yes/no proposition.

03

Scoring

How urgent?

Use an anchored rubric.

04

Endpointing

Has the turn ended?

Judge the available prefix.

Probability and intensity answer different questions

NOUL · TYPESAFE SEMANTICS

Was cancellation requested?

A probability-valued
yes/no judgment.

An uncertain answer does not mean
“a medium amount of cancellation.”

SCORE · ILLUSTRATIVE RUBRIC

How urgent is the issue?

Can wait

Work disrupted

Critical service blocked

Endpointing is a decision in time

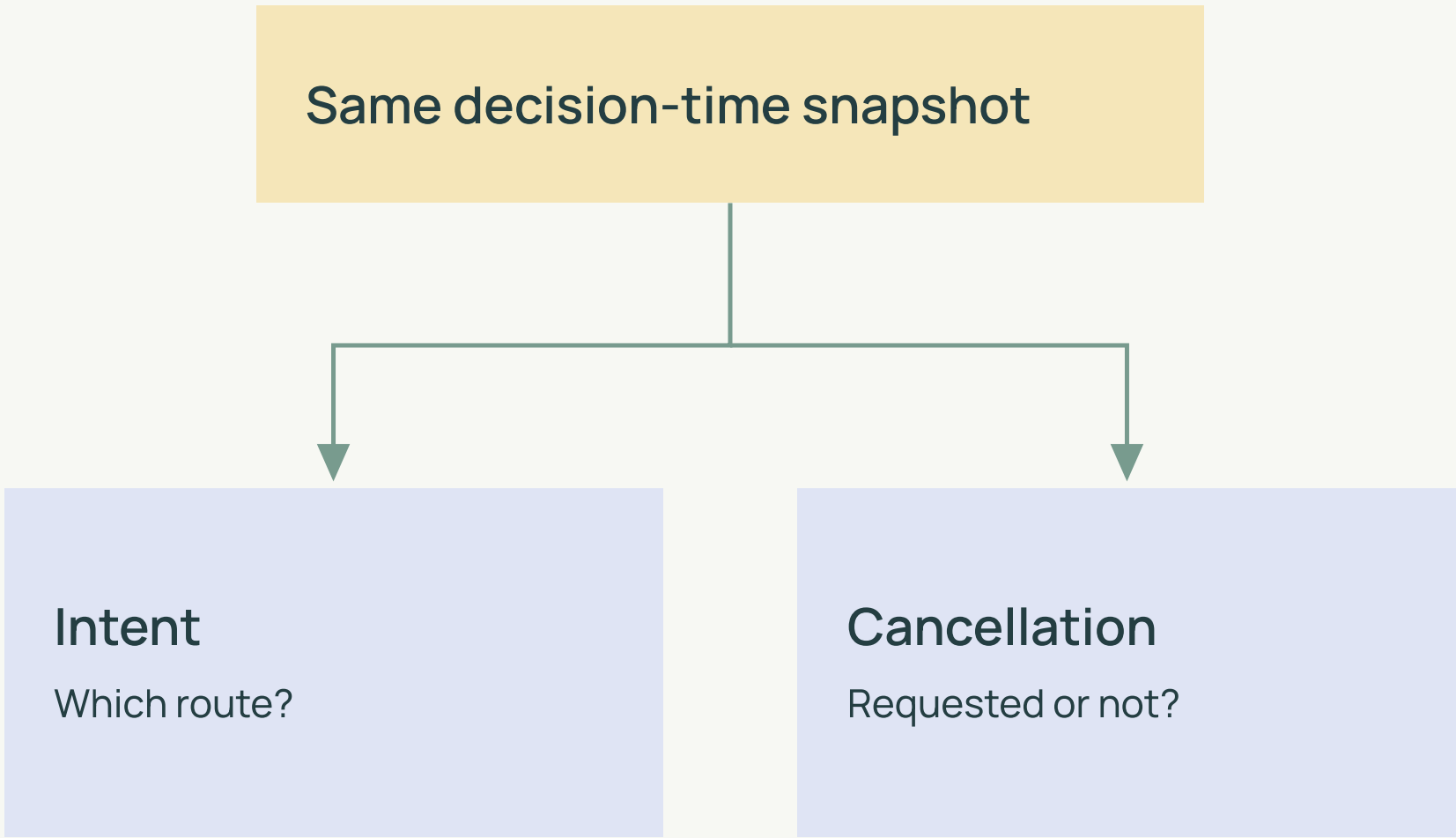


Cut too early
The customer is interrupted.

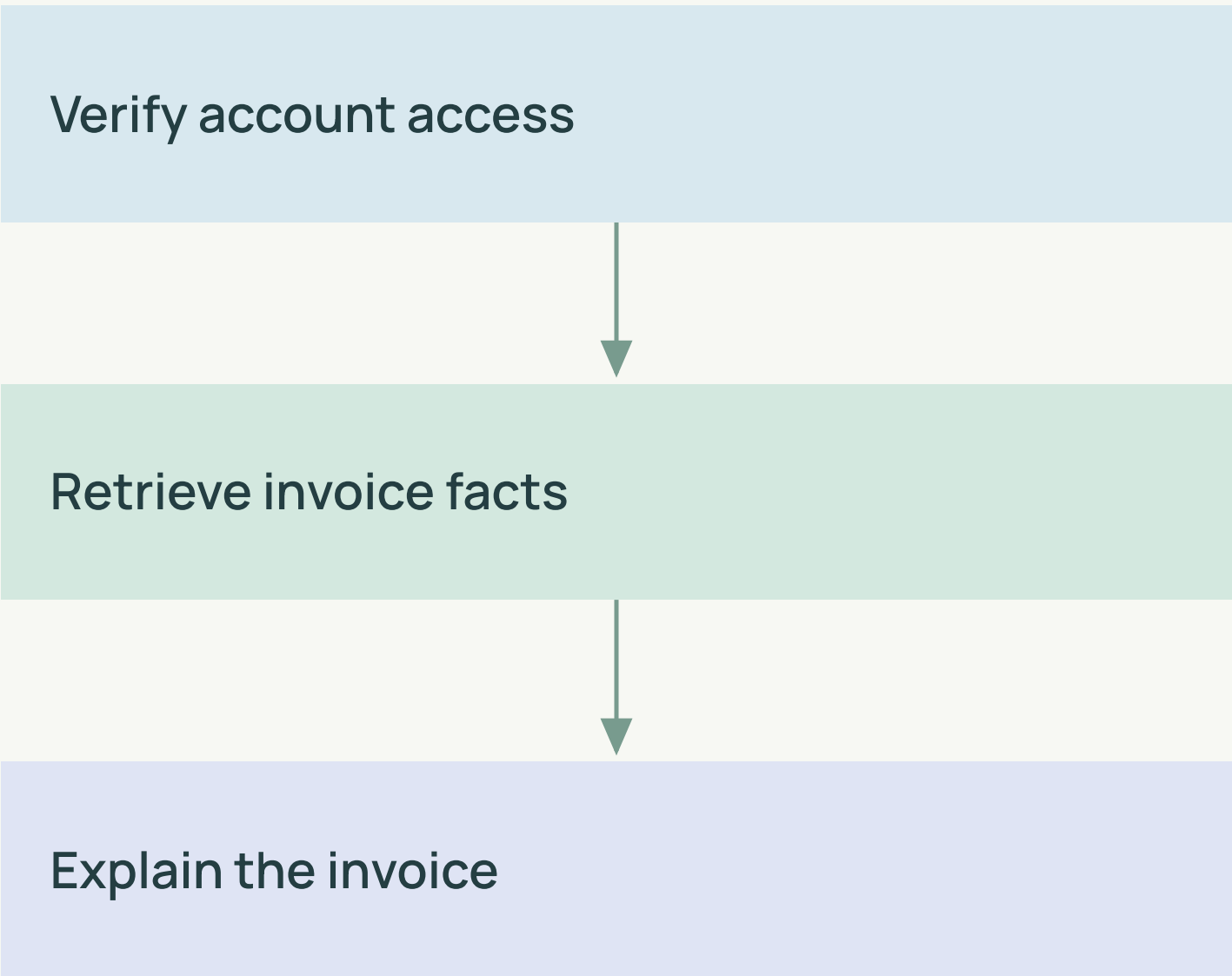
Wait too long
The response is delayed.

Keep the boundary
Future audio may label the prefix.
It cannot enter the input.

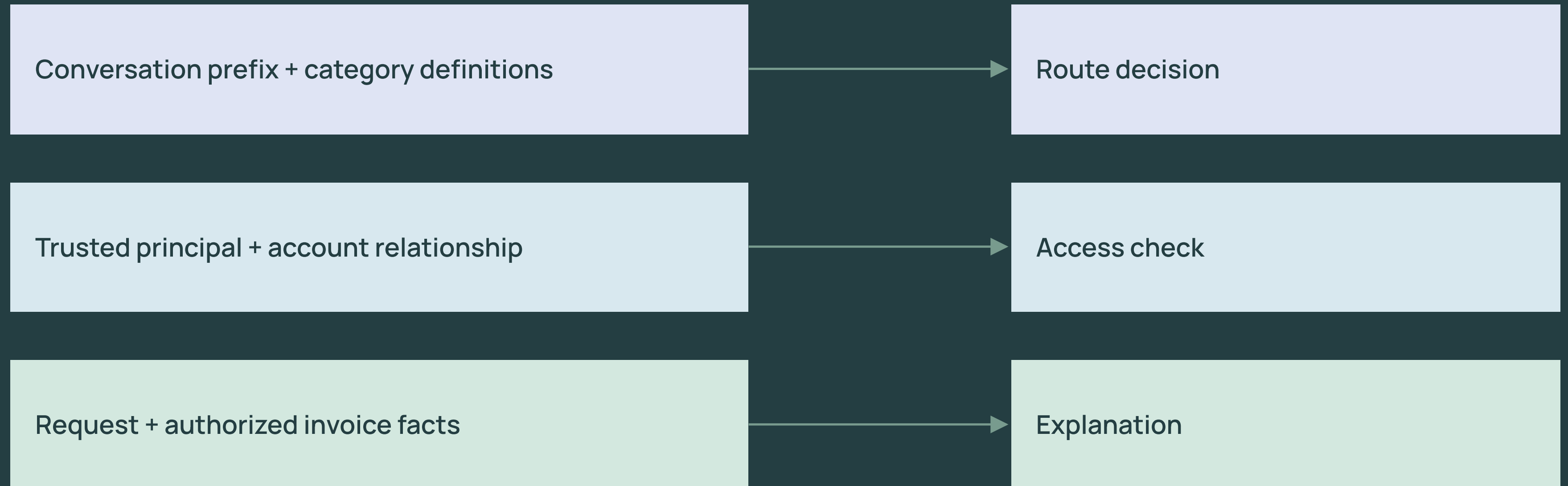
Split work along information dependencies



Independent execution can still produce correlated errors.



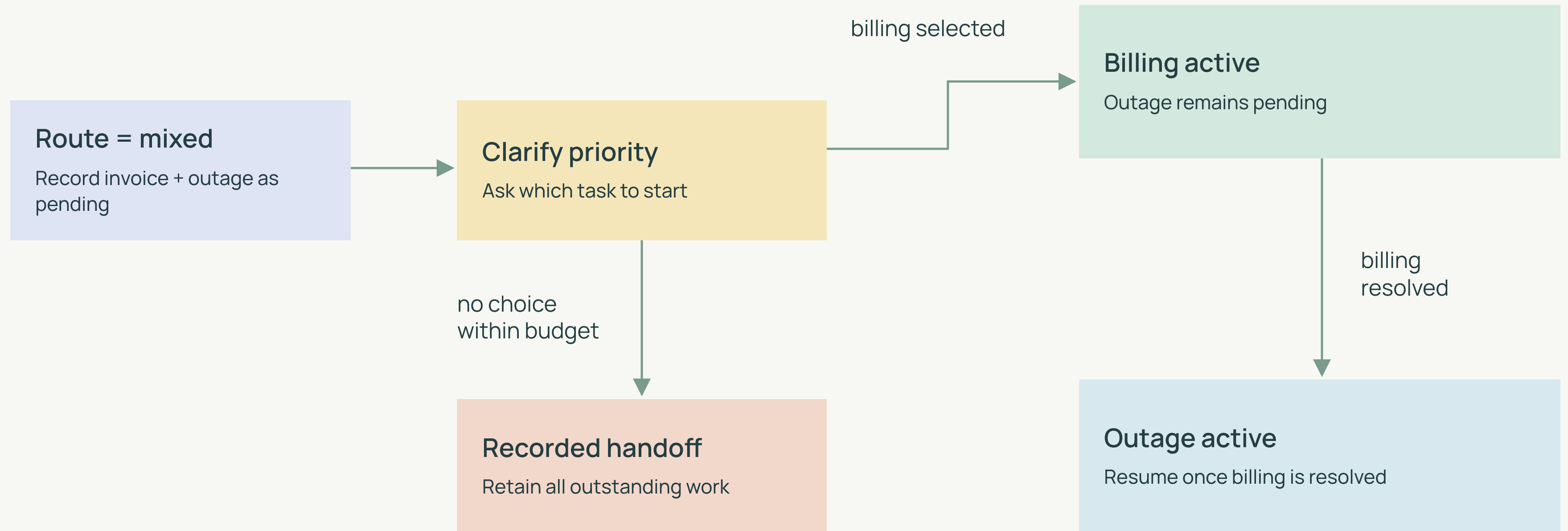
Each node sees a different view of the case



A node receives its declared input view; task state remains in the graph.

04 / STEP 3

State preserves work across a branch



Guided determinism gives uncertainty a place

Neural interpretation inside an authored process.

AgentScript defines where model choice is available.
Agent Graph executes the configured transitions.

Interpretation

Fallible semantic judgment.

Procedure

Required steps and state changes.

Authority

Enforced by trusted services.

Availability and required execution differ

DESIGN PSEUDOCODE

Control around the model choice.

```
access = checkAccess(trustedAccount)
if access != allowed: go to recovery
facts = loadInvoice(trustedAccount)
offer explain(facts) when factsReady
keep outage.pending
```

Backend services also enforce access.
This is an authoring sketch, not runnable syntax.

REQUIRED WORK

`run` / procedure

Use explicit procedure logic for checks that must execute.

PERMITTED CHOICES

`available when`

Filter which actions the model may select. Availability does not force selection.

CONTROLLED ARGUMENTS

`with` / bound input

Bind consequential inputs from trusted state.

A billing path preserves the outage task



Technical support accepts the handoff; the outage remains unresolved.

INVOICE

Resolved

Resolution requires customer confirmation.

OUTAGE

Handed off

Accepted handoff preserves ownership. It still leaves work unresolved.

Event 8 of 8

Failure routes are part of the program

CONTRACT FAILURE

Invalid output

Validate the result before branching. Use a defined fallback.

DEPENDENCY FAILURE

Timeout

Bound retries and retain every pending obligation.

AUTHORITY BOUNDARY

Unauthorized write

The service rejects the action under current policy.

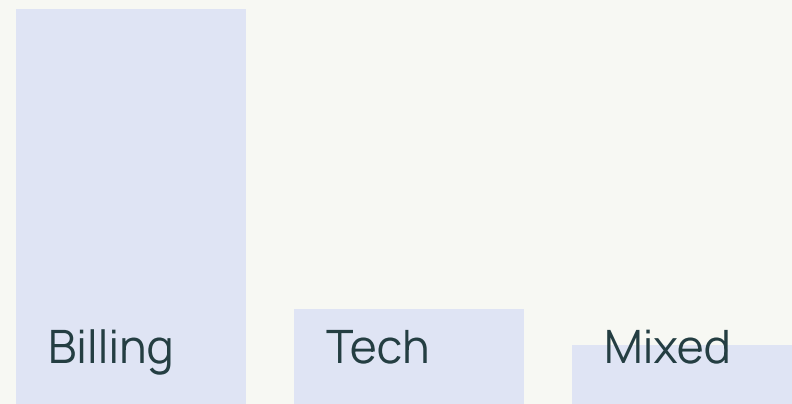
SIDE EFFECT

Repeated write

Use transaction safeguards; record the actual result.

Confidence needs evidence in your domain

A concentrated distribution



Illustrative output shape.
The highest option can still be wrong.

VALIDATE LOCALLY

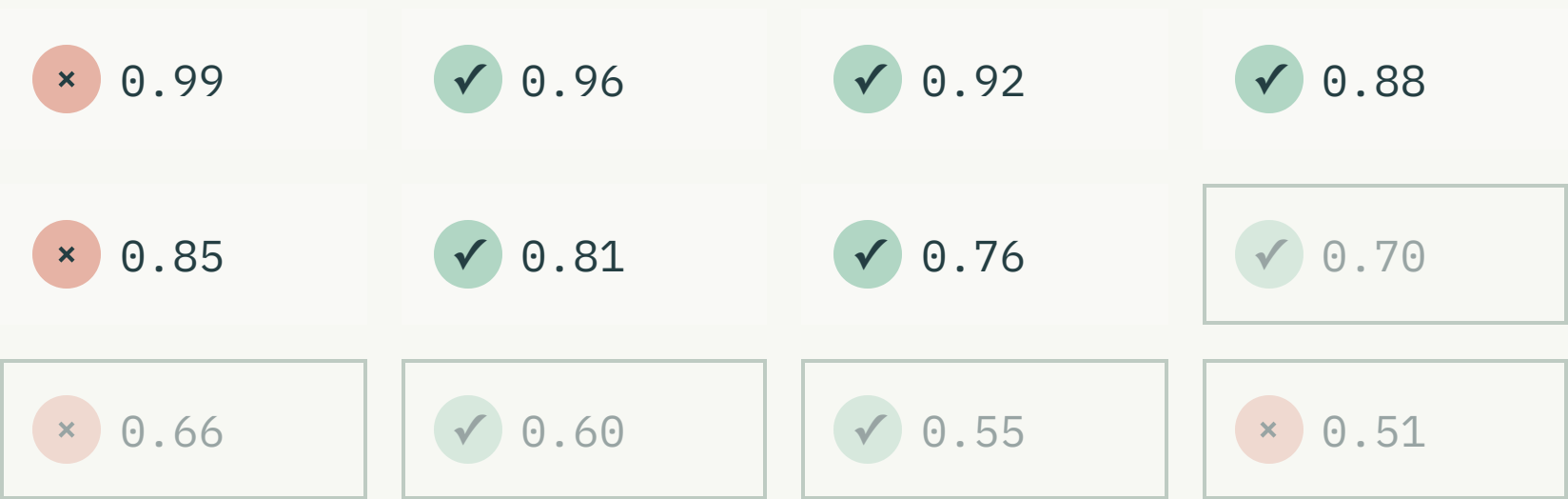
Compare signals
with adjudicated labels.

Choose gates on development data.
Report error alongside coverage.

Identity, permissions, and policy
remain separate checks.

A stricter gate moves work somewhere

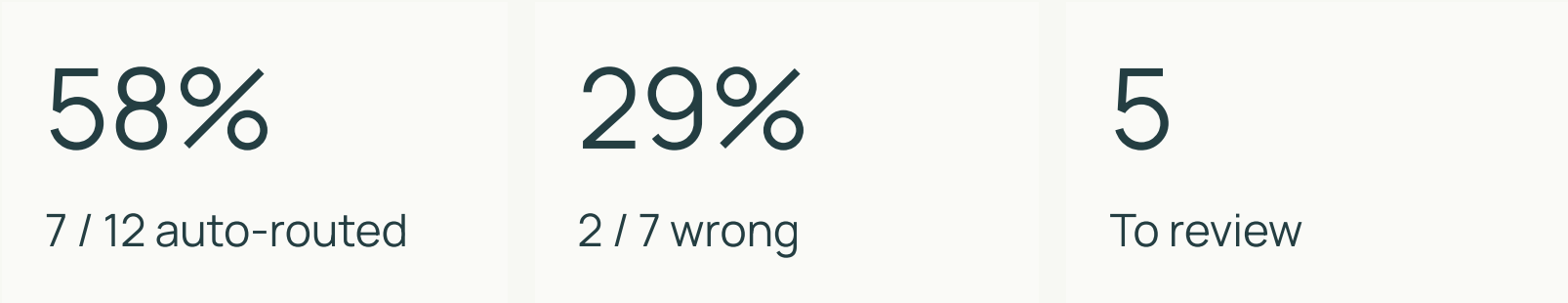
Twelve constructed examples



✓ correct route · ✗ wrong route
Faded examples go to review.

SYNTHETIC SAMPLE / UNCALIBRATED SIGNAL

Automatic-route threshold: 0.75



A high-scoring mistake can survive the gate.
Review consumes capacity and can add delay.

Savings depend on avoiding expensive work

ROUTING STEP ONLY · INVENTED UNIT COSTS

Baseline = G

Candidate = D + O + pG

G = 1.00 expensive routing call
D = 0.03 specialized decision
O = 0.04 added orchestration

Routes still using G: 20%

HYPOTHETICAL ROUTING-STAGE ARITHMETIC

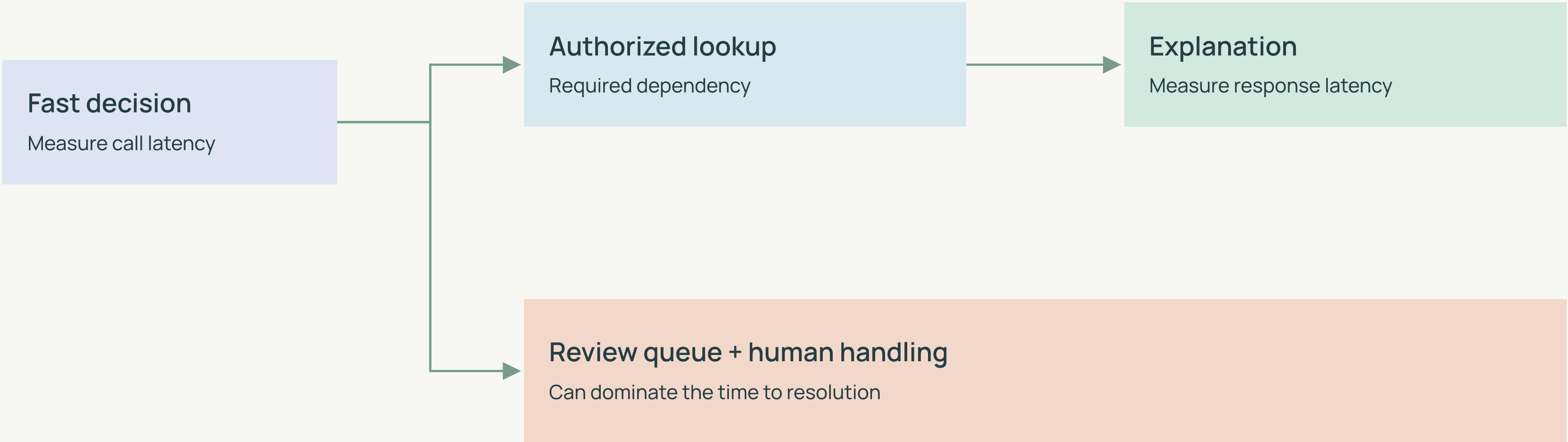


73% lower

0.27 normalized units

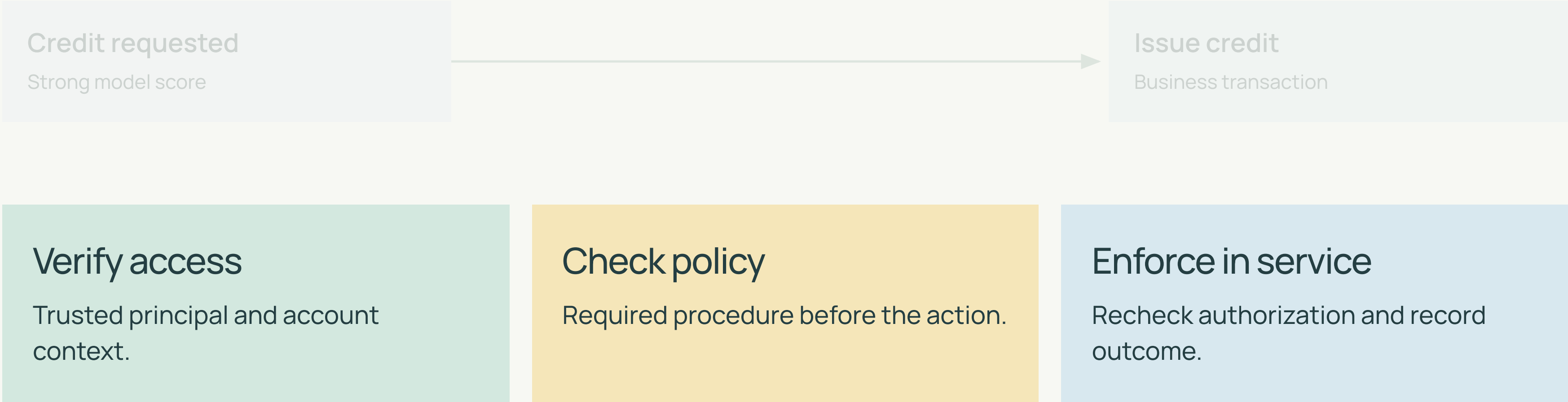
Break-even: $D + O = (1 - p)G$
Later explanation and service work excluded.

Fast inference helps the path that uses it



Track decision time, response time, and time to completed service separately.

Find the missing boundary



Begin the dataset at the decision boundary

At decision time

INPUT SNAPSHOT

Conversation prefix
Trusted facts available then
Current policy version

REFERENCE LABEL

Next route supported by that evidence.

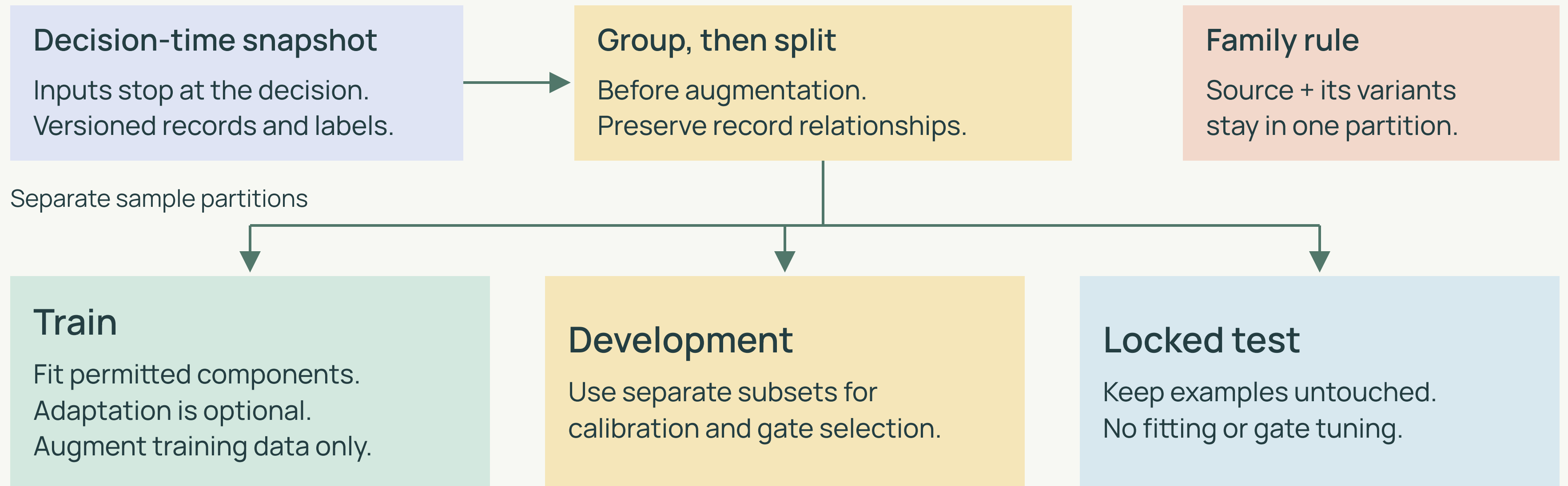
After the decision

LATER TURNS AND OUTCOMES

Useful for separate outcome labels.
Excluded from the earlier input.

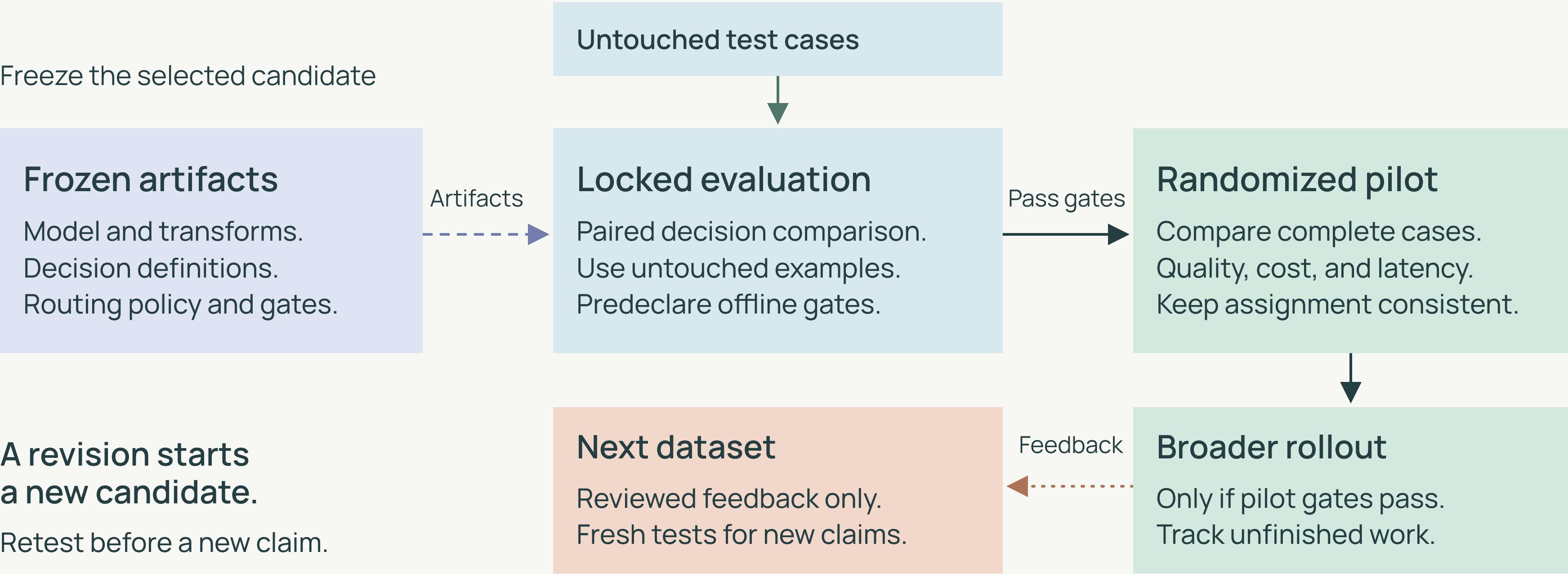
Preserve order,
relationships,
and provenance.

Split examples before developing the decision



Route labels use decision-time evidence; later outcomes belong to separate targets.

Freeze the workflow before earning release



Choose the split that matches the claim

UNSEEN ACCOUNTS

Keep account families intact.



Related cases and variants stay in one partition.

FUTURE TRAFFIC

Reserve a later-time cohort.



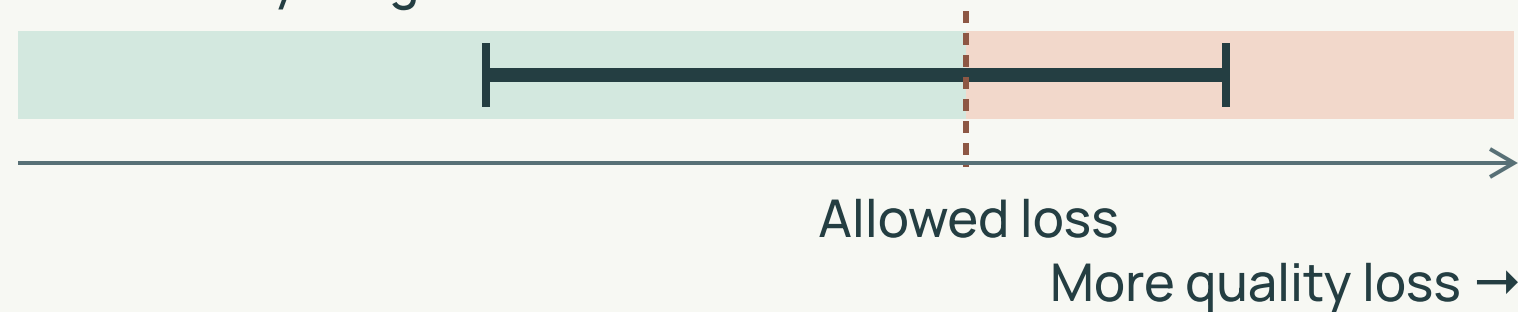
Declare boundary exclusions and duplicate handling.

Lower cost must come with acceptable quality

Schematic decision rule · no measured results

Quality: upper end must stay below the limit

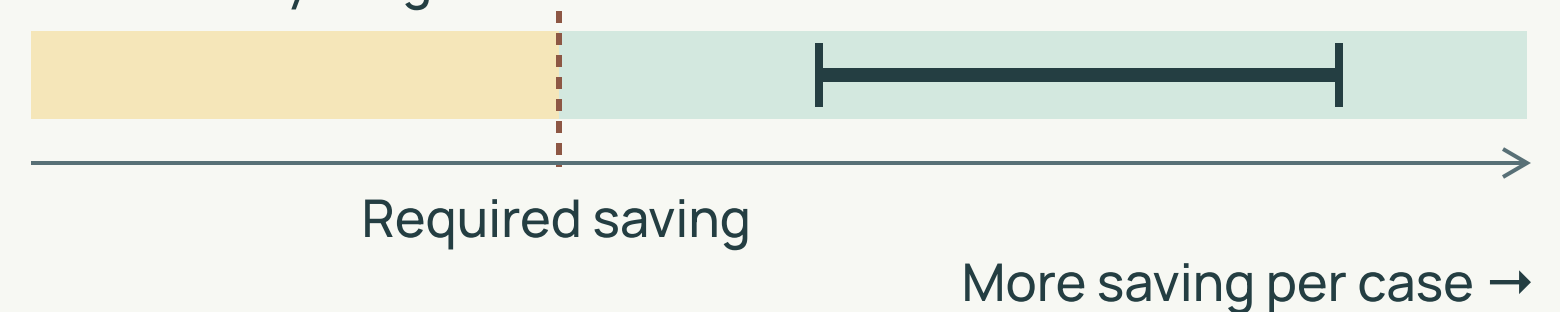
Uncertainty range



This range crosses the margin.
Quality remains uncertain.

Cost: lower end must exceed the target

Uncertainty range



This range clears the required gain.
Cost passes; rollout still waits.

Count the whole case: invoice resolved + outage pending
= incomplete at the deadline.

Harm limits must also pass.
Assess all cases and the same relevant cohort.

Latency limits must also pass.
Include unfinished cases and resolution time.

Evaluate the decision, then the service

PAIRED OFFLINE COMPARISON

Evaluate the decision.

Same held-out snapshots.
Same route question.
Fixed permissions and tools.

Each model's gate is selected
on development data.

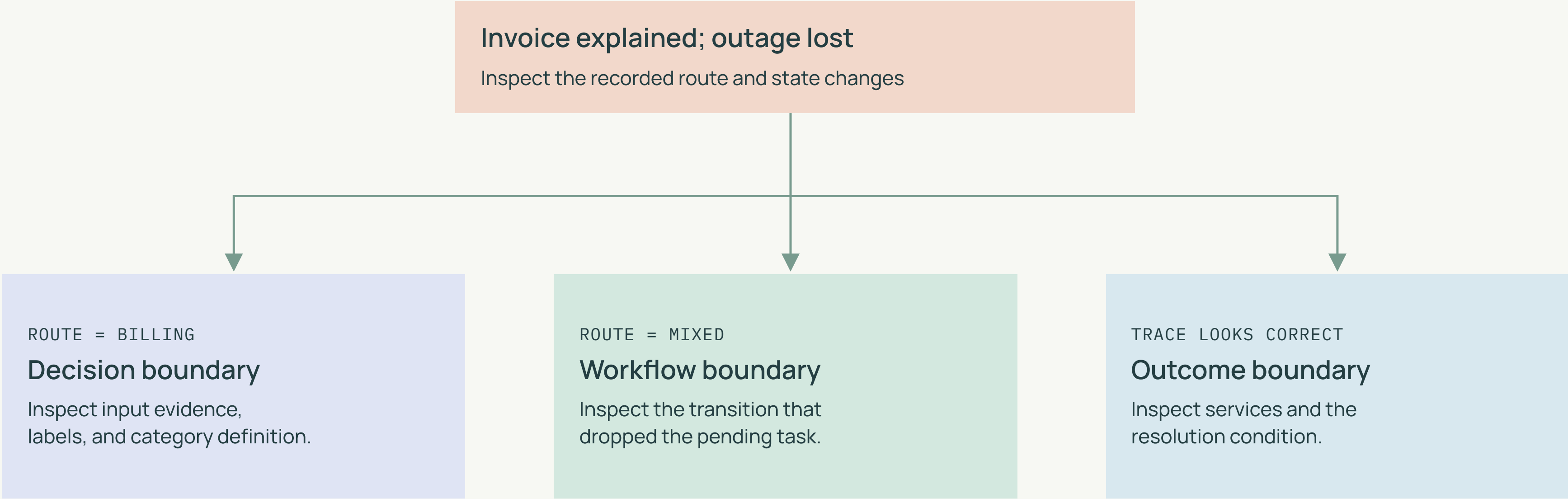
CONTROLLED SERVICE PILOT

Evaluate the complete case.

Randomized case/account assignment.
Consistent treatment and horizon.
Costs, queues, and unresolved work.

Pilot evidence must precede
wider rollout.

Diagnose the boundary that failed



Start with one decision that changes the path

Give each decision a purpose.

Give the workflow a contract.

Complete both customer requests
within the reliability and cost limits.

01 Write one bounded route question.

02 Label mixed and unclear examples.

03 Declare the node's input view.

04 Draw the guarded failure route.

05 Define complete-case acceptance.

Read the evidence behind the architecture

Salesforce control

[Agent Graph / guided determinism ↗](#)

[Reliable enterprise agents ↗](#)

[AgentScript control plane ↗](#)

[AgentScript specification ↗](#)

[All sources and claim boundaries ↗](#)

[Tutorial handout and notes ↗](#)

Agent architecture

[Jev + LangGraph ↗](#)

[A harness with Jev ↗](#)

[Jev in LangSmith Evals ↗](#)

[Jev-as-a-Judge ↗](#)

[Runtime, framework, harness ↗](#)

Decision patterns

[Composable AI ↗](#)

[TypeSafe quickstart ↗](#)

[Confidence routing ↗](#)

[Intent routing ↗](#)

[Structured decision criteria ↗](#)

[Building with System One ↗](#)

[Tutorial evaluation protocol ↗](#)

[Full training pipeline ↗](#)